

AlphaChess

How the model works, and how it is trained

*A self-play chess engine in the AlphaZero family
PyTorch · python-chess · a native C search core*

Technical documentation for the `alpha_chess` repository

Scope. This document describes the method as it is actually implemented in this repository, with the file and function names to match. Where the implementation departs from the AlphaZero paper — and it does, in several places that matter for running on one GPU — the departure is called out and justified. Numbers quoted as measurements were measured on the target host (RTX 3090, 8-core/16-thread i9-11900KF).

Contents

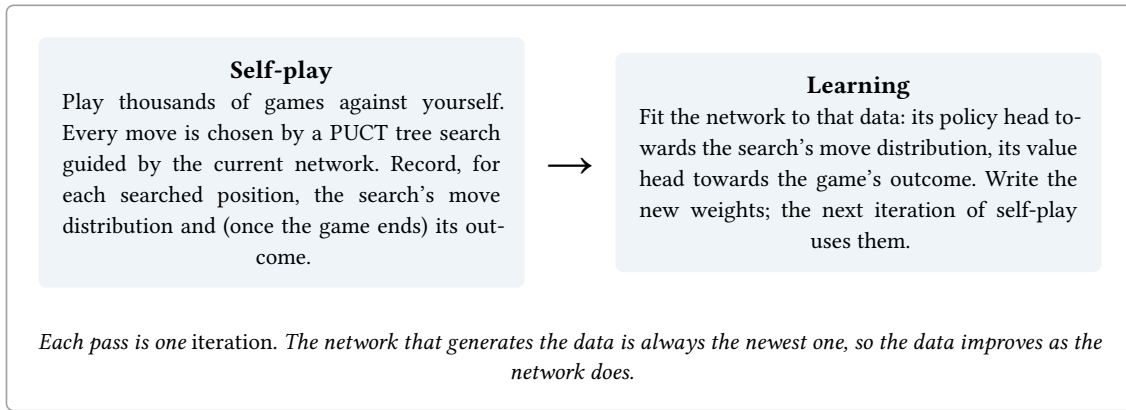
1	Overview	3
1.1	What is in this repository	3
2	Board representation	4
2.1	Side-to-move relative orientation	4
2.2	The 21 planes	4
2.3	Storage packing	5
3	Move representation	6
3.1	The 73 planes	6
3.2	Orientation, again	6
3.3	Coverage	6
4	The network	7
4.1	Architecture	7
4.2	Parameter budget	7
4.3	Why network size sets self-play speed	7
5	Search: PUCT Monte-Carlo tree search	8
5.1	What a node stores	8
5.2	The selection rule	8
5.3	A simulation	8
5.4	Terminal positions	8
5.5	Root Dirichlet noise	9
5.6	First-play urgency	9
5.7	Subtree reuse	9
5.8	Choosing the move	9
6	Self-play	11
6.1	Batching: one leaf per game per step	11
6.2	Pools: overlapping CPU and GPU	11

6.3	Playout-cap randomisation	11
6.4	Resignation	12
6.5	The native search core	12
7	Training	14
7.1	The targets	14
7.2	The loss	14
7.3	The replay buffer	14
7.4	How many gradient steps	15
7.5	Optimizer and schedule	15
7.6	Checkpoints and resuming	15
8	Measuring strength	17
9	Hyperparameter reference	17
10	What this is and is not	18

1 Overview

AlphaChess learns to play chess from nothing but the rules. There is no opening book, no endgame tablebase, no handcrafted evaluation function, and no database of human games. A single neural network is trained against data generated by searching with an earlier version of itself, and the only external signal in the entire pipeline is the result of a finished game: win, loss, or draw.

The method is a loop with two halves.



The engine of improvement is that **search is stronger than the network alone**. A network evaluated once on a position gives a decent guess at the best move; a few hundred simulations of tree search using that same network gives a better one. Training the raw network to imitate its own searched output therefore moves it towards something it could not yet produce directly — and the next search, now guided by a better network, is better still. This is policy iteration, with a neural network as the function approximator and MCTS as the policy improvement operator.

1.1 What is in this repository

<code>encoding.py</code>	board $\rightarrow$ tensor, and move $\leftrightarrow$ policy index
<code>network.py</code>	the residual policy/value network
<code>mcts.py</code>	interactive PUCT search (one position at a time)
<code>native/</code>	the C search core: bitboards, movegen, encoding, PUCT trees
<code>native_selfplay.py</code>	pooled self-play driving that core
<code>batched_selfplay.py</code>	the pure-Python self-play fallback
<code>train.py</code>	the self-play $\rightarrow$ learning loop
<code>evaluate.py</code>	strength measurement against baselines and UCI engines
<code>gui.py, agent.py, cli.py</code>	playing interfaces

The split between `mcts.py` and the native core is worth understanding early, because the two searches serve different masters. The interactive search optimises **latency**: one position, answer as fast as possible. Self-play optimises **throughput**: thousands of games at once, and no single game cares how long it waits. They implement the same algorithm and are tested against each other, but almost none of the code is shared.

2 Board representation

The network sees a position as a stack of 21 binary-or-scalar planes of shape 8×8 , i.e. a tensor of shape (21, 8, 8). This is built by `encode_board()` in `encoding.py`.

2.1 Side-to-move relative orientation

The single most important property of the encoding is that it is **relative to the side to move**, not absolute. When Black is to move, the board is flipped vertically and the piece planes are swapped so that “our pieces” always occupy planes 0–5 and always advance towards increasing rank.

The reason is sample efficiency. Chess is very nearly symmetric under colour reversal: a position and its mirror have the same evaluation with the sign flipped, and the same best move reflected. An absolute encoding forces the network to learn every motif twice — once for White, once for Black — from disjoint halves of the data. A relative encoding lets a single set of filters serve both, so every training example teaches both colours at once.

The flip is an implementation detail **contained inside** `encoding.py`. `encode_board()`, `move_to_index()` and `index_to_move()` all take the board and apply or undo the orientation themselves, so every caller works in ordinary absolute chess coordinates. Nothing outside this module has to know the network sees a flipped world.

The flip itself is a single `arr[:, ::-1, :]` on the assembled stack. Planes that are spatially constant (the castling flags, the halfmove clock, the repetition indicators, the ones plane) are unaffected by a vertical flip, so flipping everything at once is safe and costs one array copy.

2.2 The 21 planes

Planes	Values	Contents
0–5	$\frac{0}{1}$	Our pawns, knights, bishops, rooks, queens, king
6–11	$\frac{0}{1}$	Their pawns, knights, bishops, rooks, queens, king
12	$\frac{0}{1}$	Our kingside castling right (broadcast over the plane)
13	$\frac{0}{1}$	Our queenside castling right
14	$\frac{0}{1}$	Their kingside castling right
15	$\frac{0}{1}$	Their queenside castling right
16	$\frac{0}{1}$	En-passant target square (a single set square, or empty)
17	[0, 1]	Halfmove clock $\div 100$, clipped — progress toward the fifty-move draw
18	$\frac{0}{1}$	This position has occurred at least once before
19	$\frac{0}{1}$	This position has occurred at least twice before
20	1	Constant ones

Three of these deserve comment.

Planes 18–19 make the state Markovian. Chess as usually stated is not a Markov decision process on the board alone: the legality of a draw claim, and therefore the true value of a position, depends on history. Without repetition planes the network is asked to evaluate positions whose value it cannot determine from its input, and a search that finds a repetition has no way to tell the network that a draw is now available. Two planes are enough to distinguish “new”, “seen once” and “seen twice” (the third occurrence being a claimable draw).

Plane 20 lets padded convolutions find the edge. A 3×3 convolution with zero padding cannot, by itself, distinguish “the board ends here” from “there are no pieces here”. A constant plane of ones gives the

filters a signal that is 1 inside the board and 0 in the padding, which makes edge and corner effects — rook mobility, king safety on the back rank — learnable.

The piece planes are built from bitboards, not a loop. `_piece_planes()` takes python-chess’s twelve 64-bit masks, concatenates them into 96 little-endian bytes, and expands them with a single `numpy.unpackbits` call. Bit i of a mask is square i , so the unpacked bits reshape directly to `[plane, rank, file]`. This replaces a Python loop over `board.piece_map()` and is the difference between encoding being a rounding error and being a bottleneck.

2.3 Storage packing

Stored in float32, one position is $21 \times 64 \times 4 = 5.4$ KB, and a replay buffer of two million positions would be 10.8 GB of states alone. Every value in the encoding is either binary or a scaled small rational, so `pack_state()` quantises to uint8 and `unpack_state()` restores:

$$\text{packed} = \text{round}(x \cdot 255), \quad x = \frac{\text{packed}}{255}$$

This is lossless for the binary planes and costs at most $\frac{1}{255}$ of precision on plane 17, which is itself only a coarse progress indicator. The cost is 1.3 KB per position instead of 5.4 KB.

3 Move representation

The policy head emits a fixed vector of 4672 logits. That number is 64×73 : for each of the 64 **origin** squares, 73 possible **move types**. This is the AlphaZero move encoding, and it is chosen so that a move's index depends on geometry rather than on the particular position — the same index means the same thing everywhere on the board.

$$\text{index}(m) = \text{from} \times 73 + \text{plane}(m)$$

3.1 The 73 planes

Planes	Count	Move type
0–55	56	Queen-like slides: 8 directions (N, NE, E, SE, S, SW, W, NW) $\times$ 7 distances (1–7). Plane $= d \times 7 + (n - 1)$.
56–63	8	Knight jumps, one plane per offset.
64–72	9	Underpromotions: 3 file deltas (left, straight, right) $\times$ 3 pieces (knight, bishop, rook).

Everything that is not a knight jump or an underpromotion is a queen-like slide, including king moves (distance 1), **castling** (encoded as the king's two-square move), **en passant** (a diagonal pawn slide), and **queen promotion** (a pawn's one-square advance or capture onto the back rank). Queen promotion needs no plane of its own because a pawn reaching the back rank can only promote, and promoting to a queen is the overwhelming default; the three underpromotion planes exist precisely because that default is occasionally wrong.

3.2 Orientation, again

`move_to_index()` applies the same vertical flip as the encoder before computing the index:

```
if board.turn != chess.WHITE:
    from_sq ^= 56          # rank r -> rank 7-r, file unchanged
    to_sq   ^= 56
```

The `^ 56` trick works because a square index is `rank * 8 + file`, so XOR-ing the top three bits with 111 maps rank r to rank $7 - r$ and leaves the file alone. The consequence is the one that matters: **a position and its colour mirror map corresponding moves to identical policy indices**, so the policy head shares all its learning across colours exactly as the board encoding does.

3.3 Coverage

The encoding is not obviously complete, and `test_encoding.py` checks that it is: every legal move, across openings, random playouts, promotions, underpromotions, castling and en passant, must survive a round trip through `move_to_index()` and `index_to_move()`. Of the 4672 slots, a large fraction are never legal from any position (a knight jumping off the board, a promotion from the middle of the board); these simply never receive prior mass, because the search masks the policy to the legal moves before using it.

4 The network

AlphaZeroNet in `network.py` is a standard AlphaZero-style residual tower with two heads. It maps a batch of encoded boards to a policy and a value:

$$f_{\theta} : (B, 21, 8, 8) \rightarrow \left(\underbrace{(B, 4672)}_{\text{policy logits}}, \underbrace{(B, 1)}_{\text{value}} \right)$$

4.1 Architecture

Stage	Layers
Stem	Conv2d(21 → C, 3×3, pad 1, no bias) → BatchNorm2d → ReLU
Residual tower	B blocks, each: Conv3×3 → BN → ReLU → Conv3×3 → BN, then add the block input and ReLU. Channels stay at C and the spatial size stays 8×8 throughout — there is no pooling or striding anywhere in the network.
Policy head	Conv2d($C \rightarrow 32$, 1×1) → BN → ReLU → flatten (2048) → Linear(2048 → 4672)
Value head	Conv2d($C \rightarrow 32$, 1×1) → BN → ReLU → flatten (2048) → Linear(2048 → 64) → ReLU → Linear(64 → 1) → tanh

The defaults are $C = 128$ channels and $B = 10$ blocks. No spatial downsampling is used because the board is already tiny: an 8×8 grid after ten 3×3 convolutions has a receptive field covering the whole board several times over, and pooling would only discard the square-level detail the policy head needs.

The value head’s tanh bounds the output to $[-1, +1]$, matching the game outcomes it is trained against: +1 win, 0 draw, −1 loss, always **from the perspective of the side to move**.

4.2 Parameter budget

Component (at $C = 128$, $B = 10$)	Parameters	Share
Stem	24,448	0.2%
Residual tower (10 blocks)	2,954,240	23.3%
Policy head	9,577,088	75.5%
Value head	135,361	1.1%
Total	12,691,137	100%

Three quarters of the model is the policy head’s final linear layer, a dense 2048×4672 matrix. This is a known and slightly awkward property of the AlphaZero move encoding: the output space is large and mostly unreachable, and a fully-connected projection into it dominates the parameter count while the convolutional tower — where the chess understanding actually lives — is a quarter of the weights. It is left as-is here because it matches the reference method and because parameters are not the binding constraint; **throughput** is.

For scale, at other sizes: 96×8 is 11.1M parameters, 128×10 is 12.7M, 160×12 is 15.3M. The tower grows quadratically in C while the policy head grows linearly, so the ratio improves as the network widens.

4.3 Why network size sets self-play speed

Once the search itself is fast (§6.5), the GPU forward pass is the bottleneck of the entire pipeline, and network size translates directly into games per hour. Measured at batch 1024 on a 3090: 96×8 runs at ~ 207 k positions/s, 128×10 at ~ 121 k, 160×12 at ~ 65 k. A smaller network buys data volume at the cost of ceiling. The 128×10 default is the balance point chosen for the repository’s 1500–2000 Elo target.

5 Search: PUCT Monte-Carlo tree search

The network alone is a move-ordering heuristic and a static evaluator. What actually plays chess — and what generates the training targets — is a best-first tree search that uses the network to decide where to look.

5.1 What a node stores

Each edge (s, a) from a node holds four statistics, all from the perspective of **the player to move at s** :

$N(s, a)$	visit count — how many simulations went through this edge
$W(s, a)$	total action value — the sum of every value backed up through it
$Q(s, a)$	mean action value, W/N
$P(s, a)$	prior probability, from the network's policy at s

5.2 The selection rule

At every node on the way down, the search takes the child maximising

$$a^* = \operatorname{argmax}_a \left[Q(s, a) + c_{\text{puct}} \cdot P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \right]$$

with $c_{\text{puct}} = 1.5$ by default. The two terms trade off directly:

- $Q(s, a)$ is **exploitation** — what the search has learned this move is worth.
- The second term is **exploration**, large for moves the network likes (P high) and for moves that have not been tried (N small), and it shrinks as $1/(1 + N)$ once a move starts getting visits. The $\sqrt{\sum_b N}$ factor keeps exploration alive as the node's total visit count grows, rather than letting the first promising move monopolise the budget.

On the very first descent through a node, $\sum_b N(s, b) = 0$, the exploration term vanishes for every child, and the prior alone decides — the search's first guess is exactly the network's.

5.3 A simulation

Each simulation is four phases:

1. **Select.** From the root, apply the PUCT rule repeatedly until reaching a node that has not been expanded, pushing each move onto a scratch board as you go.
2. **Evaluate.** Run the network on the leaf. Mask its policy logits to the legal moves, softmax over just those, and use the result as the priors P of the new edges. The value head gives v , the leaf's evaluation from the leaf player's point of view.
3. **Expand.** Create the leaf's children with those priors and $N = W = Q = 0$.
4. **Back up.** Walk the path back to the root, and at each step **negate the value** before accumulating it:

$$N \leftarrow N + 1, \quad W \leftarrow W + v, \quad Q \leftarrow W/N$$

The negation is what makes this work for a two-player zero-sum game — a position that is good for the mover is exactly as bad for the opponent who chose the move leading into it.

5.4 Terminal positions

A node that is checkmate, stalemate, insufficient material, fifty-move or a threefold repetition has a value that is **known**, not estimated, and the network must not be consulted for it. Both searches resolve this once, when the node is first reached, and cache the result on the node (`terminal_value`).

This is a deliberate performance decision as much as a correctness one. Calling `python-chess's is_game_over(claim_draw=True)` at every node of every descent costs roughly $164 \mu\text{s}$ — utterly dominating a search whose whole point is to run hundreds of simulations per move. Resolving it once per node, at

the leaf, and reusing the cached answer on every subsequent visit removes that cost. `test_mcts.py` and `test_selfplay.py` both pin the fast path to python-chess’s `claim_draw` semantics across all five ways a game can end.

5.5 Root Dirichlet noise

A search that always follows the network’s priors can never discover that the network is wrong. At the root of a self-play search — and **only** there, and only during self-play — the priors are mixed with Dirichlet noise:

$$P'(s_0, a) = (1 - \varepsilon)P(s_0, a) + \varepsilon\eta_a, \quad \eta \sim \text{Dir}(\alpha)$$

with $\alpha = 0.3$ and $\varepsilon = 0.25$. This guarantees that every root move gets some prior mass and so has a chance of being explored, which is the mechanism by which self-play data can contain moves the current network would never play. Evaluation and interactive play add no noise: there, you want the engine’s best judgement, not exploration.

5.6 First-play urgency

The native search adds an option the Python one does not have. In the rule above, an unvisited child has no Q , and the implementation must supply one. The plain choice — and what `mcts.py` does — is $Q = 0$, i.e. “assume a draw”.

With `--fpu-reduction` set, the native core instead gives unvisited children the parent’s own value, discounted by how much prior mass has already been explored:

$$Q_{\text{fpu}} = Q_{\text{parent}} - r \cdot \sqrt{\sum_{a: N(s, a) > 0} P(s, a)}$$

The intuition: if the search has already looked at most of the plausible moves and found the position good, an untried move is probably also roughly that good (so inheriting Q_{parent} beats assuming a draw), but the more has already been explored without finding better, the less likely an untried move is to be an improvement (so the penalty grows). Setting $r = 0$ recovers the flat-zero behaviour exactly, which is the default.

5.7 Subtree reuse

When a move is played, the subtree under it is not thrown away — it becomes the new root, keeping its visit counts and values. The work spent evaluating the opponent’s likely replies is work that does not have to be repeated.

In the native core this is `arena_copy_subtree()`, which copies the chosen child’s subtree into a freshly reset arena rather than maintaining a free list. With an untrained network the saving is small (the visit distribution is nearly flat, so 179 of 200 simulations per ply are still new work), but it grows substantially as the network sharpens and the search starts concentrating its visits.

5.8 Choosing the move

After the simulation budget is spent, the move is chosen from the root’s **visit counts**, not from Q and not from the network’s policy. Visit counts are the search’s real output: PUCT allocates visits to moves in proportion to how good they keep looking under repeated examination, which makes N a lower-variance signal than any single node’s Q .

During the first `--temperature-moves` plies (default 30) of a self-play game, the move is **sampled** in proportion to visit count:

$$\pi(a) = \frac{N(s_0, a)}{\sum_b N(s_0, b)}$$

After that, and always during evaluation, the most-visited move is played deterministically. The early sampling is what makes self-play games diverge from one another; without it, a deterministic engine against itself would play the same game every time and the replay buffer would contain one position per ply for the entire run.

6 Self-play

Self-play is the wall-clock bottleneck of the whole method, and most of this repository’s engineering is here. The naive implementation — one game, one simulation at a time, one position per forward pass — leaves a modern GPU about 99% idle.

6.1 Batching: one leaf per game per step

The structural trick is to run many games **concurrently** and interleave their searches. Each game advances its own tree by exactly one simulation per step, and every game’s leaf position for that step is collected into a single batch:

- for each step:**
1. for every active game: descend its tree to a leaf (CPU)
 2. stack all those leaves into one ($G, 21, 8, 8$) batch
 3. one forward pass (GPU)
 4. for every game: expand its leaf and back up (CPU)
 5. any game whose budget is spent plays its move and may finish

With `--games-in-flight 1024`, the network sees batches of about a thousand positions instead of one. Since a GPU’s throughput at batch 1 and at batch 1024 differ by orders of magnitude, this single change is most of the speedup: a 128×10 network runs at $\sim 94\text{k}$ positions/s at batch 512, $\sim 121\text{k}$ at 1024, and $\sim 127\text{k}$ at 2048.

The cost is memory: each in-flight game holds its own search tree and its own accumulated training examples, so `--games-in-flight` is bounded by host RAM.

6.2 Pools: overlapping CPU and GPU

Batching alone still alternates: the CPU descends while the GPU idles, then the GPU computes while the CPU idles. `--pools` runs several independent groups of games so that one pool’s tree search overlaps the next pool’s forward pass.

Two pools suffice in principle; three is the default because it also absorbs the jitter from a pool that momentarily has few active games. More than three just fragments the batches. The knob interacts with `--games-per-iter`: a pool refills a finished game with a fresh one until its quota is exhausted, after which the batch decays as the last games drain, so keeping `games-per-iter` well above `pools  $\times$  games-in-flight` (16000 vs 3072 in the default recipe) keeps that tail a small share of the iteration.

6.3 Playout-cap randomisation

Most of a game’s search budget buys very little. The position is not close, and a 200-simulation search picks the move a 50-simulation search would. Playout-cap randomisation — from KataGo¹ — exploits this asymmetry between the two things a self-play game produces.

A game produces **policy targets** (one per searched ply, and only worth recording if the search was strong) and **one value target** (the game’s result, which is worth exactly as much regardless of how carefully the moves were chosen). So:

Ply type	Treatment
Full (probability <code>--full-search-prob</code> , default 0.25)	Full <code>--simulations</code> budget, root Dirichlet noise, and the resulting visit distribution is recorded as a training example.

¹Wu, D. J. (2019). *Accelerating Self-Play Learning in Go*.

Ply type	Treatment
Fast (the other 75%)	Only --fast-simulations (default 50), no noise, no training example recorded . These plies exist purely to move the game along to a result.

Measured over 400 complete games, this takes a ply from 179 evaluated positions to 77 — so the same GPU-hour finishes **2.2× as many games**, and therefore 2.2× as many game outcomes to learn a value function from, while every policy target it records still comes from a full-strength search. Subtree reuse compounds it: a fast ply often already has more carried-over visits than its budget calls for, and costs a single simulation.

The accounting consequence is that an iteration records roughly 4× fewer positions per game, which the --sample-reuse rule (§7.4) accounts for automatically.

6.4 Resignation

Playing out a hopeless position to checkmate is wasted GPU time. When the mover’s best root value stays at or below --resign-threshold (default −0.90) for two consecutive plies, the game is resigned and scored as a loss.

The danger is obvious: a resignation threshold that is too aggressive mislabels positions that were actually saveable, and poisons the value target. So a fraction of games — --resign-disable-fraction, default 10% — is played out with resignation **suppressed**, purely so the engine can measure how often it **would** have resigned a game it went on to draw or win. That false-positive rate is printed every iteration. If it climbs above about 5%, the threshold is too loose and should be lowered.

6.5 The native search core

Profiling the pure-Python engine put the time as follows:

Where self-play time went (Python engine)	Share
python-chess legal move generation	~40%
Move → policy index	~14%
Board → tensor encoding	~10%
Tree descent (push, PUCT select, backup)	~17%

None of that is GPU work. All of it is Python doing bookkeeping, and the card sat idle waiting for it. So the board, move generation, the encoding and the PUCT tree were all moved into a small C extension (alpha_chess/native/):

bitboard.h	attack tables and runtime-generated magic bitboards
position.h	position, Zobrist hashing, make-move, legal move generation
encode.h	the 21-plane encoding and 4672-way index, matching encoding.py
mcts.h	arena-allocated PUCT trees and the self-play game driver
fastchess.c	CPython bindings; releases the GIL for the whole search step

The extension is compiled on first use — native.py tries cc, gcc, clang, then the pip-installed ziglang — so no system compiler and no build step is required of the user, and a host without one silently falls back to the Python engine.

Result. ~ 102k evaluated positions/s against ~ 27k for the pure-Python engine on the same hardware: **3.8× the raw search throughput**, and with playout caps, **8.5× the games per hour**. The bottleneck moves off the CPU and onto the GPU, which is where it should be — the card now runs at its power limit. A week-long run becomes a long weekend.

The obligation this creates is correctness: a second chess implementation is a second chance to be subtly wrong. `test_native.py` runs **perft** on the six standard positions (~194M nodes between them, covering castling, en passant, promotion, discovered check, pins and stalemate) and diffs the native legal-move set, move-to-index mapping and encoded planes against `python-chess` and `encoding.py` over thousands of positions from random games — including en-passant positions specifically, the one feature where the two encoders could plausibly disagree.

7 Training

7.1 The targets

Every recorded example is a triple (s, π, z) :

s	the encoded position, uint8-packed
π	the search's normalised root visit distribution at that position — the policy target
z	the game's eventual result, +1/0/-1, from the perspective of the player to move at s — the value target

The sign convention on z is the detail that is easy to get wrong. The game produces one White-relative result; each example then flips it according to whose turn it was at that position (`out_values[o] = turn ? result : -result`). This has to match the value head's convention — side-to-move relative, like the encoding — or the network learns the negation of chess.

Note what is **not** in the target. There is no per-move evaluation, no engine annotation, no search score. The value target is the raw, extremely noisy outcome of one game. Its saving grace is that it is unbiased: averaged over enough games from similar positions, it converges on the true value, and the network's job is precisely to do that averaging.

7.2 The loss

$$\mathcal{L} = - \underbrace{\sum_a \pi(a) \log p_\theta(a | s)}_{\text{policy: cross-entropy}} + \underbrace{(z - v_\theta(s))^2}_{\text{value: squared error}} + \underbrace{\lambda \|\theta\|^2}_{\text{weight decay}}$$

The policy term is a cross-entropy between the search distribution and the network's softmax, which pulls the raw network towards what the search concluded. The value term is a plain MSE against the game result. Weight decay (10^{-4}) is applied by the optimizer rather than added explicitly.

The two terms are summed with **equal weight**. There is no tuning constant between them, matching AlphaZero; the value loss is naturally smaller in magnitude and the policy term dominates the gradient early in training.

The policy target is sparse. A 4672-wide dense float32 target per example would be 18.7 KB, most of it zeros — only the moves that actually received visits are non-zero, at most `MAX_POLICY_TARGETS = 80` of them. So the buffer stores (index, value) pairs and the loss gathers rather than materialising the dense row:

```
log_probs = F.log_softmax(policy_logits, dim=1)
picked = log_probs.gather(1, pol_idx)          # (B, 80)
policy_loss = -(pol_val * picked).sum(dim=1).mean()
```

Padded slots carry value 0 and so contribute nothing. This is mathematically identical to the dense cross-entropy and vastly cheaper in both memory and bandwidth.

7.3 The replay buffer

A fixed-capacity ring of examples (`ReplayBuffer` in `self_play.py`), default two million positions. Training samples uniformly from it, so each iteration learns from a **window of recent history** rather than only the games just generated — which decorrelates consecutive gradient steps and stops the network chasing the idiosyncrasies of one batch of games.

The storage is column-oriented and deliberately compact, because buffer size is one of the levers on final strength and a dense buffer runs out of RAM long before it runs out of usefulness:

-- resume pointed at a directory or a `train_state.pt` restores all of that **and** the replay buffer, and continues from the next iteration. Pointed at a plain model checkpoint instead, it loads weights only and starts a fresh optimizer — useful for fine-tuning. Missing optimizer state never crashes the run.

8 Measuring strength

`evaluate.py` plays a checkpoint against an opponent over a series of games with alternating colours and reports W/D/L, score, and an Elo difference:

$$\Delta\text{Elo} = -400 \log_{10} \left(\frac{1}{S} - 1 \right)$$

where S is the score in $[0, 1]$, clamped at $1/(2n)$ so a whitewash gives a finite estimate that tightens as the match lengthens.

Opponent	What it tells you
random	Sanity only. Any trained network should crush it.
material	A greedy material-counting baseline. The first real rung.
model:PATH	Another checkpoint of your own — measures improvement .
uci:PATH	An external engine (e.g. Stockfish) — the only path to a comparable number.

The Elo number is relative to the opponent, not absolute. Beating material by 300 points says nothing about FIDE rating. Only a calibrated external engine anchors a comparable figure — and even then, note that engines enforce a floor on strength limiting (Stockfish 16's UCI_Elo will not go below 1320; below that, use `--uci-skill`, which reaches Skill Level 0), and that a strength-limited engine is not a human of the same nominal rating. It plays weaker moves but still never hangs a piece.

Games are decorrelated by `--opening-plies` random opening moves per colour-swapped pair, so two deterministic players still produce varied games rather than repeating one line.

9 Hyperparameter reference

The defaults below are the RTX 3090 recipe from the README.

Flag	Default	Role
<code>--iterations</code>	80	Self-play $\rightarrow$ train cycles; also the cosine LR horizon
<code>--games-per-iter</code>	16000	Games generated per iteration
<code>--simulations</code>	200	Budget on full-search plies
<code>--fast-simulations</code>	50	Budget on playout-capped plies
<code>--full-search-prob</code>	0.25	Fraction of plies searched fully and recorded
<code>--pools</code>	3	Independent game pools, for CPU/GPU overlap
<code>--games-in-flight</code>	1024	Concurrent games per pool = the network batch
<code>--channels / --blocks</code>	128 / 10	Network width and depth
<code>--batch-size</code>	1024	Minibatch for the gradient step
<code>--epochs</code>	4	Optimization passes per iteration
<code>--sample-reuse</code>	4.0	Expected samples per position per lifetime
<code>--lr</code>	10^{-3}	Initial Adam LR, cosine-decayed to 10^{-4}
<code>--weight-decay</code>	10^{-4}	L2 regularisation
<code>--grad-clip</code>	1.0	Max gradient norm
<code>--buffer-size</code>	2,000,000	Replay capacity in positions (~ 3.8 GB)
<code>--temperature-moves</code>	30	Plies sampled from visit counts before going greedy
<code>--max-moves</code>	400	Ply cap; a cut-off game is scored a draw
<code>--resign-threshold</code>	-0.90	Root value at which a game is resigned
<code>--resign-disable-fraction</code>	0.10	Games played out to measure resign false positives

Flag	Default	Role
- - c -puct (internal)	1.5	PUCT exploration constant
Dirichlet α, ε	0.3, 0.25	Root noise shape and mixing weight

10 What this is and is not

This is a faithful, runnable implementation of the AlphaZero method, engineered so that the loop closes on a single consumer GPU in days rather than on a TPU pod in hours. The reference work used 5,000 first-generation TPUs for self-play and 64 second-generation TPUs for training; the target here is one RTX 3090 and a practical ceiling around 1500–2000 Elo.

Everything that makes that feasible is a throughput decision, not an algorithmic shortcut: batching thousands of concurrent games into one forward pass, moving the search into C so the GPU is the bottleneck, spending the search budget where it buys game outcomes rather than uniformly, and storing the replay buffer compactly enough that a useful window fits in RAM. The learning algorithm itself — PUCT with network priors, visit-count policy targets, game-outcome value targets, and a combined cross-entropy plus MSE loss — is unchanged.

What it is not is a promise of grandmaster strength. The scaling is real but so is the gap: AlphaZero’s published result is several orders of magnitude more compute away.